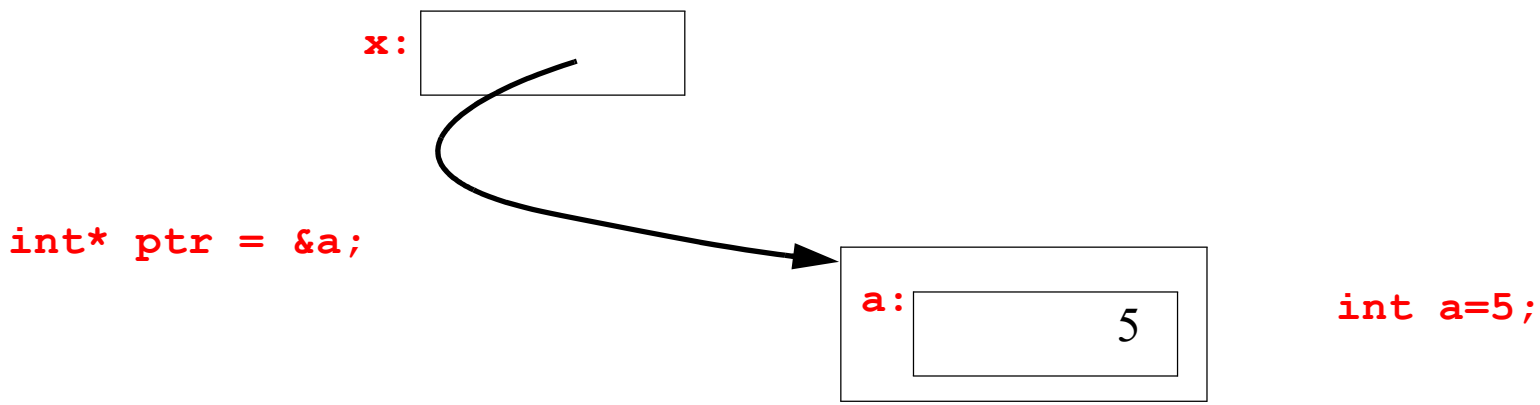
Week 4

- Memory Management

- Thanks to Paul Kube for the slides

Pictures of memory

- It is useful to abstract the essentials about the contents of memory by drawing simple pictures...
- Here is a picture of the contents of memory after those statements (either the Java or the C++ version) execute:
  - The pointer variable `x` points to the object, an instance of the class `C`, which contains an instance variable named `a` with value 5:



  - Always be aware of the difference between a pointer and what it points to!

Memory

Memory distribution in a typical operating system process

- Stack – the stack is for local variables and for maintaining a record of function calls. The stack grows from the top of memory down towards the heap.
- Heap - The heap is for dynamically allocated data items. The heap grows from the top of the static data area up as data items are allocated.
- Static Data - This is a block of reserved space in RAM for all the global and static variables from your program. Allocated once and lasts for duration of a program.

Differences between Java and C++

- Pointers:
  - In Java, declaring a variable of class type creates a pointer that can point to an instance of that type (an object)
  - in C++, you have to explicitly specify that the variable is a pointer by using `*` after the typename

  - Dereferences:
  - In Java, the dot operator `.` dereferences a pointer and accesses a member of the object pointed to
  - in C++, the `->` operator does that; or, you can use a combination of the dereference operator `*` and the member access operator `.` That is, these are equivalent in C++: `x->getA()` and `(*x).getA()`

Memory management in Java and C++

- In Java, all objects are created using **new**, and memory an object takes is automatically reclaimed by the garbage collector when it can no longer be accessed in your program:

```
C x = new C(); // create an object
x = null;   // object is inaccessible; no problem, gc will free
```

- In C++, objects can be created using **new**, but memory for such objects is not automatically reclaimed:

```
C* x = new C();  // create an object using new
x = nullptr;     // object is inaccessible;
                 // that's a problem, it will not be freed
```

- That is a *memory leak*! In C++, an object created using new must be explicitly freed using **delete** for its memory to be reclaimed for later use in your program

```
C* x = new C();    // create an object using new
...                // do things with the object
delete x;          // free the object using delete
x = nullptr;       // make pointer null to avoid dangling pointer
```

- Note: some objects created in other ways in a C++ program *are* reclaimed automatically. Understanding memory management is important in C++ programming

Automatic, static, and dynamic data in Java and C++

- A running program typically has 3 kinds of data in it:
  - automatic
  - static
  - dynamic

- These kinds of data differ in:
  - what region of machine memory the data is stored in
  - when and how memory for the data is allocated ("created")
  - when and how memory for the data is deallocated ("destroyed")

- Both Java and C++ have basically these same 3 kinds of data, with some differences

- (Note: these are kinds of data, i.e. variables. Executable instructions are stored somewhat differently.)

Automatic data

- Automatic data is called "automatic" because it is automatically created and destroyed when your program runs
  - Examples of automatic data in both Java and C++: formal parameters to functions, local variables in functions (if the variables are not declared static)
- Memory for automatic data is allocated on the runtime stack
- Memory for automatic data is allocated when execution reaches the scope of the variable's declaration
- Memory for automatic data is deallocated when execution leaves that scope

Static data

- Static data is called "static" because it essentially exists during the entire execution of your program
  - Examples of static data in Java and C++: variables declared static (or, in C++, variables declared outside any function and class body)
- Memory for static data is allocated in the static data segment
- Memory for static data is allocated when your program starts, or when execution reaches the static data declaration for the first time
- Memory for static data is deallocated when your program exits

Dynamic data

- Dynamic data is called "dynamic" because it is allocated (and, in C/C++, destroyed) under explicit programmer control
  - Example of dynamic data in Java: *all* objects (instances of a class), including arrays. Dynamic data is created using the **`new`** operator
  - Example of dynamic data in C++: any variables created using the **`new`** operator. In C++, any type variable can be created using **`new`**
- Memory for dynamic data is allocated from the "heap"
- In Java, dynamic data exists from the time **`new`** is called until the object is reclaimed by the garbage collector
- In C++, dynamic data exists from the time it is allocated with the **`new`** operator until it is deallocated with the **`delete`** operator

Example of dynamic objects in C++

- Consider this C++ function. The object created with **new** is dynamic and must be explicitly destroyed with **delete**. Other variables (**i**, **j**, **x**) are automatic.

```
int foo(int i) {
   int j = i + 1;
   C* x;
   x = new C();
   x->setA(j);
   j = x->getA();
   delete x;
   return j;
 }
```

Example of automatic objects in C++

- Now consider this C++ function.
- Here `x` refers directly to an object, an instance of the class `C` (not a pointer!), created automatically on the stack, and initialized with the default constructor. It will automatically be destroyed when the function returns. Other variables (`i`,`j`) are also automatic.

```
int foo(int i) {
   int j = i + 1;
   C x;
   x.setA(j);
   j = x.getA();
   return j;
}
```

- Note the use of the ordinary C++ member access operator `.`
- Note that Java does not permit creating objects as automatic data on the stack like this

Dynamic data bugs

- Automatic and static data is created and destroyed automatically. As a programmer, you should know how and when this happens, but you do not need to take steps to make it happen
- Because the standard C++ runtime environment does not have a garbage collector, both allocation and deallocation of *dynamic* data must be done under programmer control
- This leads to common bugs that can be very hard to track down.
- We'll look at these types of bugs:
  - Inaccessible objects
  - Memory leaks
  - Dangling pointers

Inaccessible objects

- The inaccessible object bug: changing the value of the only pointer to an object, so you can't access the object anymore

```
thing* p = new thing();

thing* q = new thing();

p = q;       // the first thing is now inaccessible
```

Memory leaks

- Memory leaks: forgetting to delete dynamic data (often related to inacessible objects)
  - (these are hard to get in Java, because of its garbage collector)

```
void foo() {
   double* q = new double(3.0);
   /* stuff, but no delete q */
}



...


for(i=0;i<1000000;i++) foo();   // massive memory leak!
```

Dangling pointers

- A "dangling pointer" is a pointer variable that contains a non-null address that is no longer valid... the pointer isn't null, but it isn't pointing to a valid object either
  - (Hard to get in Java, with no pointers to automatic variables, and good garbage collection)

```
int* bar() {
   int i;        // automatic variable, allocated on stack
   return &i;    // return pointer to automatic variable...
                 // bad news!  Stack frame is popped upon return
}
...
   int* p;
   int* q;

   p = new int(99);

   q = p;

   delete p;              // q and p are now dangling pointers
   p = 0;                 // q is still dangling!
```